



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELÉCTRICA

CARRERA DE SOFTWARE

Asignatura: Aplicaciones Informáticas II

Tema: Diseño arquitectónico de la aplicación V3

Fecha de última actualización: 10 – 05 – 2026

Nombre:

Andrés Sebastián Peralta Ramos (7356)

Período académico:

Marzo 2026 – Julio 2026

1. Introducción

1.1. Propósito

El presente documento define la arquitectura de software para la aplicación web de gestión de perfiles académicos mediante microcredenciales e insignias digitales en la Escuela Superior Politécnica de Chimborazo (ESPOCH). El documento establece las decisiones arquitectónicas, los componentes del sistema, las vistas arquitectónicas y los patrones de diseño implementados. Este documento sirve como referencia técnica para el equipo de desarrollo durante la implementación, mantenimiento y evolución del sistema. Asimismo, proporciona a los stakeholders institucionales una visión clara de la estructura técnica de la solución.

La audiencia objetivo incluye:

- **Equipo de desarrollo:** Responsables de la implementación y pruebas del sistema
- **Director del proyecto:** Para evaluación del cumplimiento técnico
- **Personal del Decanato de Teleeducación de la ESPOCH:** Clientes institucionales del sistema

1.2. Alcance

El sistema consiste en una aplicación web basada en arquitectura cliente-servidor en tres capas que permite la gestión integral de microcredenciales e insignias digitales bajo el estándar Open Badges 3.0. La plataforma facilita la emisión, validación, almacenamiento y verificación de credenciales académicas digitales para estudiantes y docentes de la ESPOCH.

El alcance funcional abarca:

- Autenticación y gestión de usuarios con roles diferenciados (Administrador, Emisor, Receptor)
- Registro y aprobación de microcredenciales con metadatos estructurados

- Emisión de insignias digitales con firma criptográfica RS256
- Visualización y gestión de perfiles académicos digitales
- Verificación pública de credenciales mediante URLs persistentes

El sistema se integra mediante APIs REST utilizando JSON-LD para el intercambio de datos. La capa de presentación se desarrolla en Angular, el backend en Node.js con Express, y la persistencia de datos en PostgreSQL.

2. Alcance del sistema

2.1. Alcance funcional

El sistema incluye las siguientes funcionalidades:

Módulo de autenticación y gestión de usuarios

- Inicio de sesión seguro con correo institucional y contraseña
- Restablecimiento de contraseña mediante verificación
- Edición de perfil personal
- Gestión de usuarios (CRUD) por parte del Administrador
- Administración de roles y permisos (RBAC)

Módulo de gestión de microcredenciales

- Registro de microcredenciales con metadatos Open Badges 3.0
- Diseño y asociación de insignias digitales
- Aprobación o rechazo de microcredenciales por el Administrador
- Búsqueda, filtrado y modificación de estado
- Eliminación lógica preservando historial

Módulo de emisión y validación de insignias

- Emisión de insignias con firma criptográfica RS256

- Consulta de historial de emisiones
- Revocación de insignias
- Visualización de insignias emitidas
- Verificación pública mediante URL persistente

Módulo de perfil académico

- Visualización de insignias obtenidas
- Búsqueda y filtrado de insignias
- Exportación en formatos PNG y JSON-LD
- Agrupación por categorías personalizables
- Edición de datos personales
- Exportación de perfil en PDF
- Visualización pública de perfiles y microcredenciales

2.2. Alcance técnico

Arquitectura

- Frontend: Angular con TypeScript
- Backend: Node.js con Express.js
- Base de datos: PostgreSQL
- Autenticación: JWT con RBAC
- Firma digital: Algoritmo RS256
- Encriptación: AES-256 para datos sensibles
- Protocolo: HTTPS obligatorio

Estándares

- Open Badges 3.0 (1EdTech)

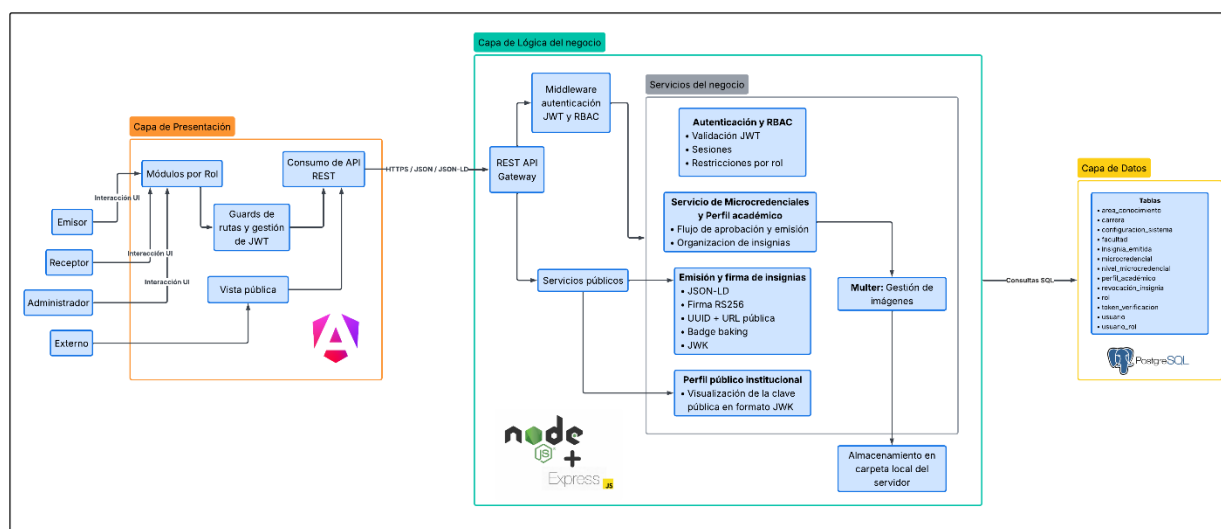
- ISO/IEC 25010:2023
- JSON-LD para metadatos

2.3. Limitaciones

El sistema no contempla la integración en tiempo real con sistemas académicos existentes, la provisión de infraestructura de hardware, una aplicación móvil nativa ni autenticación mediante sistemas externos (SSO institucional).

3. Vistas arquitectónicas

Esta sección describe la estructura del sistema mediante diferentes perspectivas, correspondiendo a la división en capas (Presentación, Lógica de Negocio y Datos) observada en el diagrama arquitectónico general.



3.1. Vista Lógica

La vista lógica detalla la organización de los componentes funcionales y los servicios del negocio dentro de la capa central del sistema. El sistema se compone de tres servicios principales encapsulados en la capa de Lógica de negocio:

- **Autenticación y RBAC:** Gestiona la validación de tokens JWT, el control de sesiones y la aplicación de restricciones basadas en roles. Este componente actúa como filtro de seguridad previo al acceso a los datos.
- **Servicio de Microcredenciales y Perfil académico:** Orquesta la lógica del flujo de aprobación de credenciales y la organización de las insignias. Coordina la información entre los emisores, receptores y la validación administrativa.
- **Emisión y Firma de insignias:** Componente responsable de la generación de credenciales verificables. Implementa la estructura JSON-LD, aplica la firma criptográfica RS256, genera identificadores UUID y URLs públicas, y gestiona el proceso de Badge baking (incrustación de metadatos en la imagen). Expone la clave pública en formato JWK para la verificación externa.

3.2. Vista de procesos

La vista de procesos define la interacción y el flujo de comunicación entre las capas del sistema:

- **Interacción Cliente-Servidor:** Los actores interactúan con la capa de Presentación. La comunicación entre la presentación y el servidor se realiza mediante el protocolo HTTPS utilizando formatos de intercambio de datos JSON y JSON-LD a través del REST API Gateway.
- **Flujo de seguridad:** Las peticiones entrantes son interceptadas por el middleware de autenticación, el cual valida la presencia y validez del token JWT antes de permitir el acceso a los servicios de negocio.
- **Gestión de archivos:** Para la carga de imágenes de insignias, se utiliza el módulo Multer, que redirige el archivo físico hacia una carpeta de almacenamiento local dentro del servidor, manteniendo la referencia lógica en el servicio de emisión.

3.3. Vista de desarrollo

La vista de desarrollo muestra la estructura de los módulos de software y las tecnologías implementadas en cada capa:

Frontend (Capa de Presentación)

- Desarrollado en Angular.
- Estructurado en módulos diferenciados por rol (incluyendo el Panel Admin).
- Implementa Guards de rutas para la protección de navegación en el lado del cliente.
- Incluye un servicio de consumo de API REST y gestión local de tokens JWT.

Backend (Capa de Lógica del negocio):

- Desarrollado en Node.js con el framework Express.js.
- Organizado mediante un patrón de arquitectura REST con un API Gateway centralizado.

3.4. Vista física

La vista física mapea los componentes de software en los nodos de hardware donde se ejecutan, según el diagrama de capas:

- **Nodo cliente (capa de Presentación):** Navegador web donde reside la aplicación Angular y donde los usuarios finales ejecutan las interacciones con el sistema.
- **Nodo servidor de aplicaciones (capa de Lógica del negocio):** Servidor que ejecuta el entorno Node.js. Contiene la lógica de negocio, los servicios de firma, la gestión de autenticación y el almacenamiento local de imágenes.
- **Nodo servidor de datos (capa de Datos):** Servidor que aloja el motor de base de datos relacional PostgreSQL, donde residen las tablas del sistema y se ejecutan las consultas SQL enviadas desde la capa lógica.

3.5. Vista de datos

La vista de datos describe el esquema de persistencia utilizado por el sistema. El almacenamiento se gestiona mediante PostgreSQL y se estructura a través de las siguientes entidades (tablas) identificadas en el diagrama:

- **Entidades principales:** usuario, microcredencial, insignia_emitida, perfil_academico.
- **Entidades de contexto:** area_conocimiento, carrera, facultad, nivel_microcredencial, rol, usuario_rol.
- **Entidades de configuración del sistema:** configuracion_sistema, revocacion_insignia, token_verificacion.

El flujo de datos indica que la capa de Lógica del negocio realiza consultas SQL directas hacia esta estructura para persistir y recuperar la información.

4. Patrones y estilos arquitectónicos

Esta sección documenta los estilos y patrones arquitectónicos implementados en el sistema, justificando su selección en función de los requisitos técnicos, la estructura en tres capas definida en el diagrama de arquitectura y las restricciones de interoperabilidad impuestas por el estándar Open Badges 3.0.

4.1. Estilo arquitectónico principal

El sistema se fundamenta en la arquitectura Cliente-Servidor en tres capas. Este estilo distribuye las responsabilidades en capa de Presentación, capa de Lógica del negocio y capa de Datos. La comunicación se establece mediante el protocolo HTTPS y el intercambio de payloads JSON/JSON-LD a través de un gateway REST. La adopción de este estilo permite la independencia tecnológica entre capas, facilita la localización de fallos, posibilita el

escalamiento horizontal del backend sin modificar la capa de presentación y cumple con los requisitos establecidos.

4.2. Patrones de diseño de software

4.2.1. Arquitectura en capas

Se aplica una separación estricta de responsabilidades. La capa de presentación gestiona la interfaz de usuario, el enrutamiento y la validación local mediante guards. La capa de negocio centraliza el flujo de dominio, las reglas de validación, la generación de credenciales y la coordinación de servicios. La capa de datos se restringe exclusivamente a la ejecución de consultas SQL y la persistencia relacional, sin contener lógica de dominio.

4.2.2. REST

La exposición de funcionalidades se estructura mediante recursos identificados por URI, utilizando verbos HTTP estándar. El servidor mantiene un estado sin sesión, delegando la gestión de contexto de autenticación al token JWT. Las respuestas se normalizan en códigos de estado HTTP y estructuras JSON consistentes.

4.2.3. Patrón middleware

En el backend se implementa una cadena de procesamiento de solicitudes. El middleware de autenticación intercepta las peticiones entrantes, valida la integridad del token JWT y evalúa los permisos RBAC antes de transferir el control a los controladores de negocio. Este patrón centraliza la lógica transversal y evita la duplicación de código en cada endpoint.

4.3. Implementación de la seguridad

4.3.1. JWT

Se implementa autenticación stateless mediante JSON Web Tokens. El token transporta claims de identidad y rol, firmado con clave secreta para garantizar integridad. Su uso elimina la dependencia de sesiones en servidor y reduce la carga de memoria en el backend.

4.3.2. RBAC

El control de acceso se gestiona mediante roles predefinidos. Las restricciones se aplican a nivel de ruta en el backend mediante middleware de autorización y se reflejan en el frontend mediante guards de navegación que ocultan o bloquean módulos no permitidos.

4.3.3. RS256

Para la emisión de insignias, se aplica el patrón de prueba criptográfica exigido por Open Badges 3.0. Cada documento JSON-LD se firma mediante un algoritmo asimétrico RS256. La clave privada permanece aislada en variables de entorno del servidor, mientras la clave pública se expone en formato JWK para verificación externa, garantizando autenticidad e integridad sin comprometer secretos.